

K A S U

Security Audit Report

RWA Private Credit Lending Platform — v3

Auditor: Claude by Anthropic (AI-Assisted Review)

Date: April 24, 2026

Commit: `33b1383cc80840b7ee166a7d41e7a2280e2e5706`

Branch: master

Solidity Version: 0.8.23

Scope: 69 contracts · ~11,630 LoC · Full + Lite deployments

AI-Assisted Security Review — Not a Replacement for Professional Audit

[kasu.finance](#) · [Docs](#) · [Twitter/X](#) · [Discord](#) · [Telegram](#) · [Medium](#)

I M P O R T A N T N O T I C E

Disclaimer

Disclaimer

This security audit was performed by Claude (Anthropic), an AI assistant. While AI-assisted reviews can identify many classes of vulnerabilities and provide valuable security insights, this report has inherent limitations:

- AI models may miss subtle vulnerabilities that require deep domain expertise or novel attack vectors.
- This review is based on static analysis of source code. No dynamic testing, formal verification, or fuzzing was performed.
- Cross-contract interactions and deployment-specific configurations may not be fully captured.
- This report should be treated as a supplement to, not a replacement for, a professional security audit by an established firm.

The Kasu team is strongly encouraged to engage a professional audit firm (e.g., Trail of Bits, OpenZeppelin, Cyfrin, Spearbit) for a comprehensive review before mainnet launch or significant upgrades.

All findings reflect the auditor's assessment at the time of review and are subject to the constraints above.

N A V I G A T I O N

Table of Contents

[Executive Summary](#)[Scope & Methodology](#)[Assumptions & Trust Model](#)[Findings Summary](#)[Low Severity Findings](#)[Informational Findings](#)[Security Checklist](#)[Go / No-Go Recommendation](#)

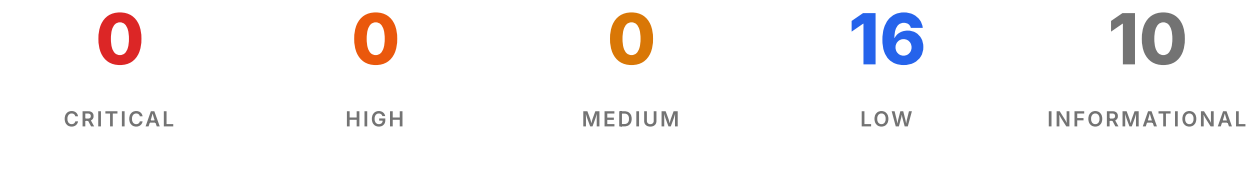
Appendix A: Contract Inventory

SECTION 1

Executive Summary

Overall Risk Assessment: LOW

The Kasu protocol demonstrates strong security engineering: comprehensive role-based access control, proper use of OpenZeppelin v5, epoch-based design that eliminates most MEV vectors, and defense-in-depth patterns throughout. This audit was conducted using the Nemesis methodology (Feynman deep logic analysis fused with State Inconsistency structural analysis in an iterative feedback loop). **No Critical, High, or Medium findings were identified.** All findings below are hardening recommendations and defense-in-depth improvements.



Key Strengths

Access control — OZ `AccessControlUpgradeable` with clean separation between Kasu admin and per-pool manager/admin multisigs.

Reentrancy surface — External calls restricted to trusted USDC / tranche / FT / KSU contracts and an admin-curated swap-exchange allowlist. CEI pattern respected.

ERC4626 share math — Tranche vault uses `_decimalsOffset = 12` to neutralize first-depositor donation / inflation attacks.

Epoch-batched clearing — Deposit/withdraw requests queue across the epoch and settle together; no per-tx auction surface for MEV.

Upgradeability — `TransparentUpgradeableProxy` with dedicated `ProxyAdmin` per proxy; every constructor in scope calls `_disableInitializers()`.

Priority Areas (All Low)

Admin-trust surface on `KSULocking` and `UserLoyaltyRewards` — narrow the blast radius of an admin multisig compromise with bounded multipliers, a deactivate-path for misconfigured lock periods, idempotency on batch emission, and forced user-receiver on emergency withdrawal. (NM-02, NM-03, NM-08)

KSU oracle hardening — add staleness, deviation, and zero-value checks at epoch rollover. (NM-05)

Config-setter gating during clearing — fee-rate and performance-fee setters lack the `isClearingTime()` guard that `setLoyaltyThresholds` already has. (NM-06)

Rounding drift and round-up overshoot — Step 4 per-user round-up and LOWER-branch fixed-rate interest rounding accumulate dust. (NM-01, NM-10)

SECTION 2

Scope & Methodology

Scope

All Solidity implementation contracts in the `src/` directory and vendored dependencies in `vendor/`, covering all deployment modes (Base Full, Plume Lite, XDC Lite, XDC-USDC Lite).

Category	Files	Representative Contracts
Core — Lending Pool	11	<code>LendingPool</code> , <code>LendingPoolTranche</code> , <code>LendingPoolManager</code> , <code>FixedTermDeposit</code> , <code>PendingPool</code> , <code>LendingPoolFactory</code>
Core — Clearing	5	<code>ClearingCoordinator</code> , <code>ClearingSteps</code> , <code>PendingRequestsPriorityCalculation</code> , <code>AcceptedRequestsCalculation</code> , <code>AcceptedRequestsExecution</code>
Core — Config & Accounting	12	<code>SystemVariables</code> , <code>UserManager</code> , <code>UserLoyaltyRewards</code> , <code>FeeManager</code> , <code>KsuPrice</code> , <code>KasuAllowList</code>
Core — Swap	2	<code>DepositSwap</code> , <code>Swapper</code>
Locking System	5	<code>KSULocking</code> , <code>rKSU</code> , <code>KSULockBonus</code>
Token & Access Control	5	<code>KSU</code> , <code>KasuController</code> , <code>KasuAccessControllable</code> , <code>Roles</code> , <code>KasuPoolExternalTVL</code>
Lite Variants	5	<code>userManagerLite</code> , <code>UserLoyaltyRewardsLite</code> , <code>KSULockingLite</code> , <code>KsuPriceLite</code> , <code>ProtocolFeeManagerLite</code>
Shared Utilities & Interfaces	22	<code>AddressLib</code> , <code>CommonErrors</code> , <code>AssetFunctionsBase</code> , all <code>I*.sol</code> interfaces
Vendor (NexeraID)	2	<code>BaseTxAuthDataVerifier</code> , <code>TxAuthDataVerifierUpgradeable</code>
Total	69	~11,630 LoC

Methodology — Nemesis Protocol

This audit was conducted using the Nemesis protocol: a feedback-loop fusion of two complementary methodologies run in alternating passes until findings converge.

1. Feynman Deep Logic Interrogation. First-principles reasoning applied to every function across seven question categories: Purpose (why does this line exist?), Ordering (can it move up/down?), Consistency (guard on funcA but missing on funcB?), Assumptions (implicit trust of caller/data/state/time?), Boundaries (first/last/repeat/self-reference?), Return/Error (silent failures, ignored returns?), and Call Reorder / Multi-Tx (reorderable external calls, cross-tx accumulator corruption?). Finds business-logic bugs that pattern-matching misses.

2. State Inconsistency Mapping. Structural invariant analysis: map every coupled state pair (per-user balance ↔ per-user checkpoint, total ↔ sum-of-parts, cached computation ↔ input), build the mutation matrix for every state variable, and cross-check every mutation path for missing state synchronization. Parallel path comparison (transfer vs burn, partial vs full, normal vs emergency). Look for masking code (ternary clamps, `Math.min`, try/catch) that hides a broken invariant underneath.

3. Nemesis Feedback Loop. Feynman suspects become extra state-audit targets; state gaps drive Feynman re-interrogation ("why is this sync missing? what breaks downstream?"). The loop alternates until no new findings emerge in a pass.

4. Verification Gate. Every candidate finding is code-trace verified against the repository at HEAD. False positives (hidden reconciliation via modifiers/hooks, lazy evaluation, designed asymmetry, economically infeasible attacks) are eliminated before the finding reaches this report.

SECTION 3

Assumptions & Trust Model

Privileged Roles

Role	Holder	Capabilities
<code>DEFAULT_ADMIN_ROLE</code> / <code>ROLE_KASU_ADMIN</code>	Kasu Multisig (per chain)	Pause system, configure fees/thresholds, manage roles, recover tokens, block users, set reward caps, manage lock periods
<code>ROLE_POOL_ADMIN</code>	Pool Admin Multisig	Create pools, configure pool parameters, manage pool-level roles
<code>ROLE_POOL_MANAGER</code>	Pool Manager Multisig	Update draw amounts, repay funds, report losses
<code>ROLE_POOL_CLEARING_MANAGER</code>	Pool Admin Multisig	Execute clearing process
<code>ROLE_POOL_FUNDS_MANAGER</code>	Pool Manager Multisig	Manage pool fund movements
<code>ROLE_LENDING_POOL_FACTORY</code>	<code>LendingPoolFactory</code> contract	Grant <code>ROLE_POOL_ADMIN</code> at pool creation
<code>ROLE_LENDING_POOL_CREATOR</code>	Pool Admin Multisig	Call <code>createPool</code> via <code>LendingPoolManager</code>
<code>ROLE_PROTOCOL_FEE_CLAIMER</code>	Configured address	Claim accumulated protocol fees

Upgradeability

TransparentUpgradeableProxy with per-proxy `ProxyAdmin` (OpenZeppelin v5 pattern) for core singletons. UpgradeableBeacon for pool instances (`LendingPool` , `PendingPool` , `LendingPoolTranche`). All `ProxyAdmin` contracts and beacons share the Kasu multisig as their owner. Every in-scope contract calls `_disableInitializers()` in its constructor.

External Dependencies

USDC — Underlying asset for all lending pools. Trusted as non-reentrant ERC20.

NexeraID / Compilot — KYC/KYB signature gating. `NexeraIDSignerManager` contract address per chain; signature verification on-chain via vendored `BaseTxAuthDataVerifier`.

Chainsight — KSU price oracle for Full deployments (via `KsuPrice`). Lite deployments use `KsuPriceLite` (returns 0).

Wrapped native + Swapper — Deposit-with-swap flow for non-USDC deposits. Swap target restricted by admin-curated exchange allowlist.

OpenZeppelin Contracts v5.0.2 — Base contracts for access control, tokens, proxies.

Trust Model

Admin multisigs are trusted; signers are cooperatively honest under threshold. Inter-contract calls between Kasu system contracts are trusted. End users interact through `LendingPoolManager`. KYC signatures are validated on-chain. Findings below are classified within this trust boundary, or are defense-in-depth improvements that narrow the blast radius of a multisig compromise.

SECTION 4

Findings Summary

ID	Severity	Title	Area
NM-01	LOW	Step 4 pro-rata round-up overshoots the Step 3 budget	Clearing
NM-02	LOW	<code>addLockPeriod</code> accepts unbounded multipliers and has no deactivate path	Locking
NM-03	LOW	<code>emitUserLoyaltyRewardBatch</code> lacks idempotency and per-user batch-local cap	Rewards
NM-04	LOW	<code>removeUserActiveShares</code> missing <code>notPendingLossMint</code> guard	Tranche
NM-05	LOW	KSU oracle snapshot has no staleness / deviation / zero check	Oracle
NM-06	LOW	Fee-rate setters not blocked during <code>isClearingTime()</code>	Config
NM-07	LOW	System pause blocks user-initiated <code>unlock</code> of matured KSU	Locking
NM-08	LOW	<code>emergencyWithdraw</code> takes an admin-chosen <code>receiver</code>	Locking
NM-09	LOW	Last-admin <code>renounceRole</code> bricks governance	Access Control
NM-10	LOW	LOWER-branch rounding drift between <code>previewWithdraw</code> and <code>redeem</code>	Lending Pool
NM-11	LOW	<code>_postSwap</code> sweeps contract balance (not swap delta) to caller	Swap
NM-12	LOW	<code>Swapper.swap</code> consumes pre-existing <code>tokenIn</code> dust as caller input	Swap
NM-13	LOW	<code>cLaimReward</code> silent truncation hides ghost liabilities	Rewards

ID	Severity	Title	Area
NM-14	LOW	Step 4 DoS via 100-user loss-mint ceiling	Clearing
NM-15	LOW	<code>maxExcessPercentage = 0</code> misconfig silently rejects every deposit	Lending Pool
NM-16	LOW	FT withdrawal/cancel not gated by <code>verifyClearingNotPending</code>	Fixed Term
I-1	INFO	Factory role scope overreach	Access Control
I-2	INFO	Zombie pool-scoped roles after pool retirement	Access Control
I-3	INFO	<code>KSULockBonus</code> grants unbounded allowance to <code>KSULocking</code>	Locking
I-4	INFO	<code>setRewardCaps</code> has no hard upper bound	Rewards
I-5	INFO	Loyalty reward sized on pre-withdrawal balance	Rewards
I-6	INFO	<code>rKSU._update</code> not guarded at internal level	Locking
I-7	INFO	<code>FeeManager.emitFees</code> silent ecosystem→protocol redirect	Fees
I-8	INFO	Immutable <code>_ksuPrice</code> on <code>SystemVariables</code>	Oracle
I-9	INFO	Immutable <code>_controller</code> on <code>KasuAccessControllable</code>	Access Control
I-10	INFO	<code>KasuPoolExternalTVL._update</code> uses string-reason revert	Token

SECTION 5

Low Severity Findings

NM-01 **LOW** Step 4 pro-rata round-up overshoots the Step 3 budget by up to (N-1) wei per group

CONTRACT`AcceptedRequestsExecution.sol:161-175`**IMPACT**

For a `(requestTranche, priority, destinationTranche)` group with N users, Step 4 rounds each user's accepted deposit UP on a non-zero remainder. Because the group's `totalAcceptedAmount` is read from a memory snapshot and never decremented, cumulative accept can reach `floor(budget) + (N-1) wei`. The Step 3 invariant `_verifyResult` is not re-run after Step 4, so the overshoot is silent. In realistic clearing groups the overshoot is a handful of wei (negligible); only pathological 1-wei-per-user groups can skew the result meaningfully.

FIX

Track a per-group `remainingAccept` counter across the loop; cap each user's accept with `min(userAcceptedDepositAmount, requestAmountLeft, remainingAccept)`; decrement after each user. Alternatively, have only the last user in a group absorb any +1 remainder.

TEST TO ADD`test_step4_perUserRoundUp_doesNotExceedStep3Budget`

NM-02 LOW `addLockPeriod` accepts unbounded multipliers and has no deactivate path

CONTRACT

`KSULocking.sol:156-163`

IMPACT

`rKSUMultiplier` and `ksuBonusMultiplier` accept any `uint256`; once set, a lock period can never be updated or deactivated without a full implementation upgrade. An admin multisig mis-configuration at deploy or during a parameter tune remains exploitable by any user until the upgrade lands — with lock multipliers large enough, a tiny KSU lock could corrupt `eligibleRKSUForFees` and fee-per-share accounting for all legitimate holders.

FIX

(a) Add hard ceilings in the setter: `require(rKSUMultiplier ≤ 10 * FULL_PERCENT)`, `require(ksuBonusMultiplier ≤ FULL_PERCENT)`. (b) Add function `deactivateLockPeriod(uint256 lockPeriod) external onlyAdmin` that sets `isActive = false`. Past locks store `rKSUMultiplier` per-entry, so retirement affects only future locks.

TESTS TO ADD

```
test_addLockPeriod_revertsOnExcessiveMultiplier ,
test_deactivateLockPeriod_blocksNewLocks_preservesExisting
```

NM-03 LOW `emitUserLoyaltyRewardBatch` lacks `(user, epoch)` idempotency and per-user batch-local cap

CONTRACT

`UserLoyaltyRewards.sol:148-184`

IMPACT

The admin batch emission path enforces `maxRewardPerUserPerBatch` per *row*, not per unique user; supplying the same user N times in one batch credits $N \times \text{perUserCap}$ to that user (bounded only by the batch cap). There is no `(user, epoch)` processed-flag, so repeated calls for the same epoch are not rejected. The `doEmitRewards` kill-switch is honored by the single-user path but not the batch path. Today the per-batch caps and admin-only gate make this low-impact, but the structure is weaker than the happy-path cron and unnecessarily widens the blast radius of a compromised multisig.

FIX

Additive hardening:

1. Top of function: `if (!doEmitRewards) revert RewardsDisabled();`
2. Track `mapping(address  $\Rightarrow$  mapping(uint256  $\Rightarrow$  bool)) _emittedForEpoch` and revert on duplicate, applied in both emission paths.
3. Track per-user batch-local sum alongside `batchTotal`; revert if `userBatchTotal[user] + ksuReward > perUserCap`.

TESTS TO ADD

```
test_emitBatch_respectsDoEmitRewardsFalse ,
test_emitBatch_revertsOnDuplicateUserEpoch ,
test_emitBatch_revertsOnDuplicateUserWithinSameCall
```

NM-04 **LOW** `removeUserActiveShares` **missing** `notPendingLossMint` **guard****CONTRACT**`LendingPoolTranche.sol:102-122`**IMPACT**

Today's user-facing exit paths (`redeem`) all carry `notPendingLossMint` , so the invariant is preserved across the `batchMintLossTokens` window. But the invariant is maintained *at the caller layer*, not at the lowest shared mutator. Any future upgrade that adds a mutation path which decrements `userActiveShares[user]` or reshuffles `_trancheUsers` without going through `redeem` (share migration, refund path, accounting fix) would silently corrupt loss-token accounting mid-mint.

FIX

Add `notPendingLossMint` modifier to `removeUserActiveShares` . Pins the invariant at the mutator instead of distributing it across callers.

TEST TO ADD`test_removeUserActiveShares_revertsDuringPendingLossMint`

NM-05 LOW KSU oracle snapshot has no staleness / deviation / zero check at epoch rollover

CONTRACT

`SystemVariables.sol:235-258` , `KsuPrice.sol:39-41`

IMPACT

`updateKsuEpochTokenPrice` is permissionless and snapshots the Chainsight oracle reading once at epoch rollover. No timestamp staleness check, no max-deviation-vs-prior-epoch check, no zero-value revert. A transient oracle glitch at the rollover block pins `ksuEpochTokenPrice` for the whole 1-week epoch. Downstream reward computation reverts or over/underpays, bounded by the per-user / per-batch caps on the loyalty path.

FIX

(a) Oracle wrapper returns `(price, updatedAt)` ; reject if `block.timestamp - updatedAt > MAX_STALENESS` . (b) Reject if `|newPrice - priorEpochPrice| / priorEpochPrice > MAX_DEVIATION_BPS` . (c) Revert on `price == 0` in Full deployments (Lite explicitly uses `KsuPriceLite` which returns 0; gate the zero-check by chain mode).

TESTS TO ADD

`test_updateEpochPrice_revertsOnStaleOracle` ,
`test_updateEpochPrice_revertsOnExtremeDeviation`

NM-06 **LOW** Fee-rate setters are not blocked during `isClearingTime()`**CONTRACT**`SystemVariables.sol:267-279, 369-394, 446-481`**IMPACT**

Clearing is a multi-block, multi-step process. `setFeeRates`, `setPerformanceFee`, `setProtocolFeeReceiver`, `setDefaultTrancheInterestChangeEpochDelay`, `setMaxTrancheInterestRate` all lack the `isClearingTime()` guard that `setLoyaltyThresholds` has. An admin multisig that mis-times a parameter change during the 48h clearing window can desynchronize Steps 2/3 from Step 4 — different fees applied within one clearing cycle. No attacker required; operator footgun.

FIX

Add `if (isClearingTime()) revert CannotConfigureDuringClearingPeriod();` to each affected setter, mirroring `setLoyaltyThresholds`.

TEST TO ADD

`test_setFeeRates_revertsDuringClearing` and analogues for each affected setter.

NM-07 **LOW** System pause blocks user-initiated `unlock` of matured KSU locks**CONTRACT**`KSULocking.sol:202`**IMPACT**

`KSULocking.unlock` carries `whenNotPaused`. During a system pause (emergency or malicious), users cannot withdraw already-matured deposits — their own funds are held hostage until admin unpauses. Pause should never block user exit from a matured position.

FIX

Remove `whenNotPaused` from `unlock`. Keep it on `lock` / `lockWithPermit` / `emitFees` / `claimFees`.

TEST TO ADD

`test_unlock_worksWhenPaused`

NM-08 LOW `emergencyWithdraw` takes an admin-chosen `receiver` , not the original user

CONTRACT

`KSULocking.sol:241-253`

IMPACT

One `onlyAdmin` tx can transfer any user's locked KSU to any address via the free `receiver` parameter. Today this is gated by the Kasu multisig (`ROLE_KASU_ADMIN`) and is no worse than a malicious upgrade under multisig compromise. However, the `receiver` parameter is unnecessary for the stated function of the call (emergency-unlock matured deposits back to their owners), and removing it narrows the blast radius of a multisig compromise without operational loss.

FIX

Force `receiver = user` (funds return to the original locker only). If operational need for an alternate receiver exists in some scenario, add a per-user signed authorization or a `queueEmergencyWithdraw` → `executeEmergencyWithdraw` flow with a time-lock and user-callable cancel window.

TEST TO ADD

`test_emergencyWithdraw_alwaysReturnsToUser`

NM-09 **LOW** Last-admin `renounceRole` bricks governance**CONTRACT**`KasuController.sol` (inherited from OZ `AccessControlUpgradeable`)**IMPACT**

The inherited OZ `AccessControlUpgradeable.renounceRole` lets the last holder of `DEFAULT_ADMIN_ROLE` (= `ROLE_KASU_ADMIN`) renounce. If triggered — intentionally via a bad governance tx, or maliciously by a compromised signer — no address can ever grant roles again. Pool setup, role grants, pause/unpause become impossible until a full redeploy.

FIX

Override `renounceRole` to forbid renouncing `DEFAULT_ADMIN_ROLE` when it is the last holder, or adopt `AccessControlDefaultAdminRulesUpgradeable` (OZ v5) which includes this guard and a transfer delay.

TEST TO ADD`test_cannotRenounceLastAdmin`**NM-10** **LOW** LOWER-branch rounding drift between `previewWithdraw` and `redeem`**CONTRACT**`LendingPool.sol:1000-1022` (`_applyFixedRateInterests`)**IMPACT**

Computes `sharesToWithdraw` via `previewWithdraw` (rounds up) then burns `assetAmount` from `redeem` (rounds down), but decrements `userOwedAmount` by the pre-redeem `userInterestAmountDiff`. Drift $d = \text{userInterestAmountDiff} - \text{assetAmount}$ accumulates across many FT clearings at fixed rate below base rate. Eventually `availableFunds() > balanceOf(USDC, lendingPool)` by cumulative dust, and a boundary withdrawal's `safeTransfer` can revert.

FIX

Decrement `userOwedAmount` by the actual `assetAmount` returned by `redeem` , or compute `userInterestAmountDiff` from the shares-first path so both values agree exactly.

TEST TO ADD`test_applyFixedRateInterests_noAvailableFundsDrift_over1000Clearings`

NM-11 **LOW** `_postSwap` sweeps contract balance (not swap delta) to caller**CONTRACT**`DepositSwap.sol:113-141`**IMPACT**

Unconditionally transfers `balanceOf(this, outToken)`, `balanceOf(this, inToken)`, and any WETH/ETH held by the manager to `msg.sender` after a swap-deposit. Any stray funds sitting on `LendingPoolManager` (from stuck transfers, failed aborts, upgrade residue) are leaked to whoever next triggers a swap-deposit. Does not affect active user funds; only stuck dust.

FIX

Capture `balanceBefore` per token and refund only the positive delta `balanceAfter - balanceBefore` of the swap (already done for `outToken` during swap; extend to `inToken` and native). Alternatively, refund `swappedAmount - amount` explicitly.

TEST TO ADD`test_postSwap_doesNotLeakStuckBalance`**NM-12** **LOW** `Swapper.swap` consumes pre-existing `tokenIn` dust as caller input**CONTRACT**`Swapper.sol:65-98`**IMPACT**

Sets `amountsIn[i] = IERC20(tokensIn[i]).balanceOf(address(this))` *after* the `safeTransferFrom` from the caller. Any pre-existing Swapper balance of `tokenIn` is silently rolled into the swap total; the resulting excess `tokenOut` routes to `LendingPoolManager` and (via NM-11) to the caller. Stuck-dust leak, not active-fund loss.

FIX

Accept explicit caller-declared `amountsIn[]` and validate against `balanceOf(this)` after transfer, or skip the balance-capture entirely and operate only against caller-transferred amounts.

TEST TO ADD`test_swap_doesNotConsumePreExistingBalance`

NM-13 LOW `claimReward` silent truncation on underfunded treasury hides ghost liabilities

CONTRACT

`UserLoyaltyRewards.sol:203-227`

IMPACT

When `ksuBalance < userRewards[sender]`, `claimReward` clamps `amount` to `ksuBalance` and decrements both `userRewards[sender]` and `totalUnclaimedRewards` by the clamped amount. Remaining users' `userRewards` still show the originally-recorded rewards, but `totalUnclaimedRewards` no longer reflects that total. First-claimer wins race; off-chain monitors that compare `totalUnclaimedRewards` against `balanceOf(this)` incorrectly conclude "solvent" while some users have permanently unclaimable balances.

FIX

Revert on treasury shortfall instead of clamping: `if (ksuBalance < amount) revert InsufficientTreasury();` Forces admin to top up the reward contract before claims can succeed.

TEST TO ADD

`test_claimReward_revertsWhenTreasuryShort`

NM-14 **LOW** Step 4 self-heal DoS via 100-user loss-mint ceiling**CONTRACT**

```
AcceptedRequestsExecution.sol:97-106
```

IMPACT

Hard-codes `CLEARING_MINT_BATCH_SIZE = 100`. A tranche with >100 users whose pool manager forgot to call `batchMintLossTokens` after `reportLoss(_, doMintLossTokens=false)` locks Step 4: every clearing batch reverts with `LossMintingStillPending`. The coordinator's `isPastClearingTime` short-circuit only applies to Steps 1/2, so the epoch cannot be gracefully skipped — queued requests roll forward unnecessarily.

FIX

Make `CLEARING_MINT_BATCH_SIZE` governance-configurable; or drain the mint queue in a dedicated pre-Step-4 phase; or reject `reportLoss(_, false)` when the tranche's user count exceeds a threshold the operator cannot plausibly drain.

TEST TO ADD

```
test_step4_revertsWith101Users_unblocksAfterManualBatchMint
```

NM-15 **LOW** `maxExcessPercentage = 0` pool misconfig silently rejects every deposit**CONTRACT**

```
LendingPool.sol ( verifyClearingConfig )
```

IMPACT

`verifyClearingConfig` only asserts `min ≤ max`. A pool deployed with `maxExcessPercentage = 0` (valid per the current check) rejects every deposit at Step 3 silently — no diagnostic event, no revert at config time. Users queue deposits that always get refunded, with no on-chain signal pointing to the config as the cause.

FIX

```
if (clearingConfig.maxExcessPercentage == 0 && !isStopped) revert  
PoolConfigurationIsIncorrect();
```

TEST TO ADD

```
test_verifyClearingConfig_rejectsZeroMaxExcessPercentage
```

NM-16 **LOW** FT withdrawal/cancel not gated by `verifyClearingNotPending`**CONTRACT**`FixedTermDeposit.sol:704-722`**IMPACT**

`_verifyWithdrawalActionTime` is sufficient under normal `isClearingTime()`. But during a delayed clearing (`isPastClearingTime = true` , running outside the clearing window), `applyFixedTermInterests` executes in batches while `isClearingTime()` is false. With `requestEpochsInAdvance = 0` , users can flip `withdrawRequested` between batches — not currently exploitable, but a defensive gap that future refactors could turn adversarial.

FIX

Add `verifyClearingNotPending(lendingPool)` modifier to `requestFixedTermDepositWithdrawal` and `cancelFixedTermDepositWithdrawalRequest` . Single-line hardening at the function boundary.

TEST TO ADD`test_requestFixedTermWithdrawal_revertsDuringDelayedClearing`

SECTION 6

Informational Findings

I-1 INFO Factory role scope overreach

`ROLE_LENDING_POOL_FACTORY` is treated as a universal capability to grant any pool role on any pool (`KasuController.sol:63-69, 132-138`). A future factory upgrade that exposes an unprotected forwarder would permit arbitrary pool takeover. Harden by restricting the factory-invoked grant path to `ROLE_POOL_ADMIN` only, at pool-creation time.

I-2 INFO Zombie pool-scoped roles after pool retirement

`_getLendingPoolRole` hashes `(pool, role)` into a per-pool slot; retired pools leave stale grants in storage. Add `bulkRevokeLendingPoolRoles` for retirement cleanup.

I-3 INFO `KSULockBonus` grants unbounded allowance to `KSULocking`

`KSULockBonus.sol:31-36` . No revoke / reduce path exists on the bonus contract. Pull pattern (`KSULocking.pullBonus` → `KSULockBonus.release(to, amount)` with a budget) is safer than a max allowance.

I-4 INFO `setRewardCaps` has no hard upper bound

`UserLoyaltyRewards.sol:192-197` accepts any `uint256` . Consider enforcing `perUser ≤ MAX_REWARD_PER_USER_CAP` at a reasonable protocol ceiling.

I-5 **INFO** **Loyalty reward sized on pre-withdrawal balance**

Loyalty processing runs before Step 4; a user who requested a full withdrawal this epoch still earns reward proportional to their pre-withdrawal balance. May be intended “final-epoch credit” — document explicitly or subtract pending-withdrawal shares.

I-6 **INFO** **`rKSU._update` not guarded at internal level**

Non-transferability is enforced only at external `transfer` / `transferFrom` / `approve` . Internal `_update` is unguarded; safe today but fragile against future inheritance that adds `ERC20PermitUpgradeable` or similar. Override `_update` to revert when both `from  $\neq$  0` and `to  $\neq$  0` .

I-7 **INFO** **`FeeManager.emitFees` silent ecosystem→protocol redirect**

When `eligibleRKSUForFees == 0` , ecosystem-share fees redirect to protocol fees. The event emitted is indistinguishable from “no ecosystem cut configured.” Emit a distinct `EcosystemFeesRedirected(amount)` event for observability.

I-8 **INFO** **Immutable `_ksuPrice` on `SystemVariables`**

Oracle rotation requires a full impl upgrade. Convert to a storage-backed address with `onlyAdmin` setter if operational flexibility for oracle-provider changes is desired (trade-off: enlarges the admin trust surface).

I-9 **INFO** **Immutable `_controller` on `KasuAccessControllable`**

Every inheriting contract bakes the controller into its implementation. Rotating the controller requires an upgrade. Same trade-off as I-8.

I-10 **INFO** **KasuPoolExternalTVL._update** uses string-reason revert

`revert("Transfers disabled")` . Convert to custom error `TransfersDisabled()` for consistency with the rest of the codebase and cheaper revert encoding.

SECTION 7

Security Checklist

Area	Verdict	Notes
Access control	PASS	OZ <code>AccessControlUpgradeable</code> + clean separation between Kasu admin and pool admin/manager multisigs. Minor hardening in I-1, I-2, and NM-09.
Reentrancy	PASS	External calls restricted to trusted USDC / tranche / FT / pendingPool / KSU and admin-curated swapper allowlist. CEI pattern respected.
Arithmetic	PASS	Solidity 0.8.23 checked math. Rounding drift quantified in NM-01 ($\leq N-1$ wei per group) and NM-10 (dust per FT-clearing).
External calls	PASS	Swap target restricted by admin allowlist; KYC verifier vendored and deterministic; Chainsight oracle snapshot covered by NM-05. Return values handled.
Upgradability	PASS	Transparent proxy + dedicated ProxyAdmin per proxy (OZ v5). <code>_disableInitializers()</code> called in every constructor in scope. Storage layouts append-only where extended.
Oracle dependencies	NOTE	No staleness / deviation / zero checks at epoch rollover — recommended additions described in NM-05.
MEV exposure	PASS	Clearing is epoch-batched, not per-tx auction. Oracle snapshot is the only frontrun-adjacent surface (NM-05).
ERC compliance	PASS	ERC4626 <code>_decimalsOffset = 12</code> neutralizes inflation attack. ERC1155 loss receipts route through a single <code>_update</code> override. ERC20 <code>permit</code> / <code>burnFrom</code> use <code>_spendAllowance</code> correctly.
KYC / signature verification	PASS	NexeraID signatures bind chainId + nonce + blockExpiration + contract + user. Cross-chain and

Area	Verdict	Notes
		same-chain replay infeasible; nonce increments on every verify.
ERC4626 share math	PASS	<code>_decimalsOffset = 12</code> applied at tranche level. First-depositor donation attack infeasible. Share issuance rounds in favor of vault; redemption rounds down.

SECTION 8

Go / No-Go Recommendation

Recommendation: GO

No Critical, High, or Medium findings. The Kasu protocol is ready to continue operating on the current deployment targets (Base Full; Plume / XDC / XDC-USDC Lite).

The Low-severity findings below are hardening recommendations that do not block continued operation. They are worth addressing in a follow-up maintenance pass to narrow the blast radius of operational errors and to tighten the admin-trust surface before the broader KSU token launch. Informational items can be bundled into the same maintenance pass or deferred to a later hygiene cycle.

Recommended Prioritization for Follow-Up Maintenance Pass

Admin-trust surface hardening (*NM-02, NM-03, NM-08*) — bounded multipliers, deactivate-path, emission idempotency, emergency-withdraw receiver = user. Cheap to fix; closes real footguns.

Operator-facing correctness (*NM-05, NM-06, NM-07*) — oracle staleness / deviation checks, fee-rate setter gating during clearing, pause-not-blocking-unlock. Eliminates known operational footguns.

Silent overshoot and rounding dust (*NM-01, NM-10*) — tests-driven precision fixes; small diffs.

Stuck-fund leakage and accounting visibility (*NM-11, NM-12, NM-13*) — swap-deposit balance sweep, swapper dust consumption, claim-reward treasury shortfall. Narrow scope.

Edge-case DoS and silent misconfigurations (*NM-04, NM-14, NM-15, NM-16*) — single-line defensive additions at specific call sites.

Informational items (*I-1 ... I-10*) — defense-in-depth improvements, documentation, and style consistency. Bundle into the same PR or defer to a hygiene cycle.

A P P E N D I X A

Contract Inventory

Core — Lending Pool

`LendingPool.sol``LendingPoolTranche.sol``LendingPoolTrancheLoss.sol``LendingPoolTrancheMigration.sol``LendingPoolManager.sol``LendingPoolFactory.sol`

- `LendingPoolHelpers.sol`
- `LendingPoolStoppable.sol`
- `FixedTermDeposit.sol`
- `PendingPool.sol`
- `UserRequestIds.sol`

Core — Clearing

`ClearingCoordinator.sol``ClearingSteps.sol``PendingRequestsPriorityCalculation.sol`

- `AcceptedRequestsCalculation.sol`
- `AcceptedRequestsExecution.sol`

Core — Config, Accounting, KYC, Token

`SystemVariables.sol``UserManager.sol``UserManagerMigration.sol``UserLoyaltyRewards.sol``FeeManager.sol``KsuPrice.sol``KasuAllowList.sol`

- `AssetFunctionsBase.sol`
- `Constants.sol`
- `DepositSwap.sol`
- `Swapper.sol`
- `KSU.sol`
- `KasuPoolExternalTVL.sol`

Locking System

`KSULocking.sol``rKSU.sol`

- `KSULockBonus.sol`

Access Control & Shared

`KasuController.sol``KasuAccessControllable.sol``Roles.sol`

- `AddressLib.sol`
- `CommonErrors.sol`

Lite Variants

`UserManagerLite.sol``UserLoyaltyRewardsLite.sol``KSULockingLite.sol`

- `KsuPriceLite.sol`

- `ProtocolFeeManagerLite.sol`

Vendor (NexeraID)

`vendor/nexera/BaseTxAuthDataVerifier.sol``vendor/nexera/TxAuthDataVerifierUpgradeable.sol`

— End of Report —